

LAB2 REPORT

Name: Mourya Nandan

Roll No: CS23B006

QUESTION 1:

What we want to do:

- To implement a new system call `trace(int mask)` in xv6.
- This system call allows a process to specify which system calls should be traced (printed to console).
- When tracing is enabled, every invocation of a selected syscall prints the **PID, syscall name, and return value**.

Example:

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 971
3: syscall read -> 298
3: syscall read -> 0
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 971
4: syscall read -> 298
4: syscall read -> 0
4: syscall close -> 0
```

What I have done:

1) User space:

- **Makefile (UPROGS section)**
Added `_trace` so that `trace.c` is compiled into the file system and available as a user program.
- **user/user.h**
Added prototype: `int trace(int mask);`
- **user/usys.pl**
Added entry: `entry("trace");`
- generates a user-side stub so programs can call `trace(mask)`.
- **user/trace.c** (new file)
Implements the user command `trace`.

- Parses arguments (mask, command).
- Calls syscall trace(mask).
- Executes the given command using exec().

trace.c CODE:

```

1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  int
6  main(int argc, char *argv[])
7  {
8      if(argc < 3){
9          fprintf(2, "Usage: trace mask command [args...]\n");
10         exit(1);
11     }
12
13     int mask = atoi(argv[1]);
14     if(trace(mask) < 0){
15         fprintf(2, "trace: syscall failed\n");
16         exit(1);
17     }
18     exec(argv[2], &argv[2]);
19
20     fprintf(2, "trace: exec %s failed\n", argv[2]);
21     exit(1);
22 }

```

Explaining the code:

• Included libraries:

- **kernel/types.h** : defines integer types used across kernel and user programs.
- **kernel/stat.h** : defines file-related structures (common include, though not directly used here).
- **user/user.h** : gives us user-level functions and syscall stubs like exit, exec, trace, atoi, printf.

• Main function flow:

- **Check arguments:** Ensure the user typed enough inputs (mask + command).
 - If not, print a usage message and exit.
- **Convert mask:** Take the second argument (string like "32") and turn it into an integer using atoi.
- **Call trace(mask):**
 - This is the new system call you added.
 - It tells the kernel: "from now on, trace syscalls for this process according to this mask."
- **Run the command:** Call exec with the command and its arguments (e.g., grep hello README).
 - exec replaces the current process image with the new program.

- If successful, this call never returns.
- **Error handling:** If exec fails, print an error message and exit.
- **What this achieves**
 - trace.c is just a wrapper program:
 - It sets up the syscall tracing (via trace(mask)),
 - then runs another program (exec) with tracing enabled.
 - This way, you can trace **any command you pass after the mask**.

2) Kernel space:

- **kernel/syscall.h**
Assigned new syscall number: **#define SYS_trace 22**
- **kernel/proc.h**
Added field to struct proc: **int trace_mask;**
- **kernel/proc.c**
 - In allocproc(): initialized p->trace_mask = 0;.
 - In fork(): copied parent's mask to child:
 - np->trace_mask = p->trace_mask;
- **kernel/sysproc.c**
Implemented kernel handler:

```

95  uint64
96  sys_trace(void)
97  {
98      int mask;
99
100     argint(0, &mask);
101     myproc()->trace_mask = mask;
102     return 0;
103 }
```

- **kernel/syscall.c**
 - Declared:
 - extern uint64 sys_trace(void);
 - **Added to syscall table:**
 - [SYS_trace] sys_trace,
 - Added array mapping syscall numbers → names:
 - static char *syscall_names[] = {
 - [SYS_fork] = "fork",
 - [SYS_exit] = "exit",
 - ...
 - [SYS_trace] = "trace",
 - };
 - **Modified syscall() to print trace:**
 - uint64 ret = syscalls[num]();
 - p->trapframe->a0 = ret;
 - if (p->trace_mask & (1 << num)) {
 - printf("%d: syscall %s -> %ld\n",

- p->pid, syscall_names[num], ret);
- }

3) Execution Flow with Example:

Input: \$ trace 32 grep hello README

- **Shell executes trace:**
 - Arguments: mask=32, command=grep hello README.
- **Inside trace.c:**
 - Calls trace(32) → enters kernel.
 - Kernel stores p->trace_mask = 32.
 - Then calls exec("grep", &argv[2]).
- **New program (grep) runs:**
 - Calls syscalls: open("README"), then multiple read(), then close().
- **Inside syscall() in kernel:**
 - Finds syscall number.
 - Executes function (e.g., sys_read).
 - Gets return value (# of bytes read).
 - Checks trace_mask: if read is enabled, prints line:
 - 3: syscall read -> 1023
- **Output:**
 - User sees PID, syscall name, and return value.
 - value represents how many bits it has read or write according to syscall.

4) Special Case: trace 2 usertests forkforkfork

- Here, mask = 2 → means trace syscalls with number 2 (fork).
- Running usertests forkforkfork executes a stress test that forks many processes.
- Trace shows lines like:
- 8: syscall fork -> 9
- 10: syscall fork -> 11
- ...
- → Parent calls fork, return value is child's PID.
- → In child process, fork() returns 0.

Sometimes kernel also prints messages like:

usertrap(): unexpected scause 0xf pid=9

This is from trap handling when user test intentionally triggers edge cases.

5) Question and Answers:

- **Why multiple reads?**
 - Files are read in chunks, not whole at once.
 - Each read() returns size of chunk. Example: 1023, 965, 431, 0.
 - 0 means "end of file reached".

- **Why PID changes each run?**
 - PID = process ID, like a roll number.
 - Every execution creates a new process with a new PID.
- **Why does grep run even if README has no “hello”?**
 - Grep still must scan the entire file.
 - It makes the same read() syscalls.
 - Trace shows system calls, not the match result.
- **If README had “hello”?**
 - Syscall trace would look identical (same reads).
 - Only difference: grep would also print the matching line.
- **Can we use another file (text.txt)?**
 - Yes. If file exists, grep will open() and read() it. Trace will show syscalls.
 - But we want to add the file in make file where we added readme.
 - If file doesn't exist, grep fails with open -> -1.
- **What is trace(mask)?**
 - It's a new system call we added.
 - It tells the kernel *which system calls of this process should be logged*.
 - Example: a mask with bit 5 set → log syscall number 5.
- **Why do we store proc->trace_mask?**
 - Each process needs its own filter of what to trace.
 - So we add a trace_mask field inside the process structure.
 - That way, the kernel knows exactly which syscalls to log for each process separately.
- **Why modify syscall.c?**
 - Because all syscalls pass through syscall(), so this is the right place to intercept and log.
- **What does the mask mean?**
 - It's a bitmask.
 - If bit N is set, syscall number N will be traced.
 - This lets us trace only the syscalls we're interested in..
- **What is fork?**
 - fork is the syscall to create a new process.
 - The parent gets back the child's PID, and the child sees return value 0.
 - It's like photocopying the process — now there are two almost identical ones.
- **Why do read() sizes vary (1023, 965, 431)?**
 - OS delivers chunks of file depending on buffers. Exact sizes differ run to run, but total matches file size.
- **We didn't implement the trace(mask) then how will it working?**
 - Actually, we did — but in many small steps.
 - In user space, trace(mask) is just a stub that issues an ecall.
 - The syscall number tells the kernel “this is the trace request”.
 - Inside the kernel, sys_trace stores the mask in proc->trace_mask.
 - Later, the syscall dispatcher checks the mask and prints the syscall info.
 - So trace(mask) looks like a normal function, but it's really a system call.

OUTPUTS:

```

hart 2 starting
init: starting sh
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 971
3: syscall read -> 298
3: syscall read -> 0
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 971
4: syscall read -> 298
4: syscall read -> 0
4: syscall close -> 0
$ trace 2 usertests forkforkfork
usertests starting
5: syscall fork -> 6
test forkforkfork: 5: syscall fork -> 7
7: syscall fork -> 8
8: syscall fork -> 9
8: syscall fork -> 10
9: syscall fork -> 11
8: syscall fork -> 12
9: syscall fork -> 13
9: syscall fork -> 14
8: syscall fork -> 15
9: syscall fork -> 16
8: syscall fork -> 17
8: syscall fork -> 18
9: syscall fork -> 19
8: syscall fork -> 20
9: syscall fork -> 21
10: syscall fork -> 22
8: syscall fork -> 23
9: syscall fork -> 24
8: syscall fork -> 25
8: syscall fork -> 26
9: syscall fork -> 27
8: syscall fork -> 28
8: syscall fork -> 29
9: syscall fork -> 30
8: syscall fork -> 31
9: syscall fork -> 30
8: syscall fork -> 31
8: syscall fork -> 32
8: syscall fork -> 33
9: syscall fork -> 34
8: syscall fork -> 35
8: syscall fork -> 36
9: syscall fork -> 37
8: syscall fork -> 38
9: syscall fork -> 39
8: syscall fork -> 40
9: syscall fork -> 41
10: syscall fork -> 42
8: syscall fork -> 43
9: syscall fork -> 44
8: syscall fork -> 45
8: syscall fork -> 46
9: syscall fork -> 47
10: syscall fork -> 48
8: syscall fork -> 49
9: syscall fork -> 50
8: syscall fork -> 51
9: syscall fork -> 52
8: syscall fork -> 53
9: syscall fork -> 54
8: syscall fork -> 55
9: syscall fork -> 56
8: syscall fork -> 57
10: syscall fork -> 58
9: syscall fork -> 59
8: syscall fork -> 60
8: syscall fork -> 61
9: syscall fork -> 62
8: syscall fork -> 63
9: syscall fork -> 64
8: syscall fork -> 65
8: syscall fork -> 66
8: syscall fork -> 67
8: syscall fork -> -1
9: syscall fork -> -1
10: syscall fork -> -1
OK
5: syscall fork -> 68
ALL TESTS PASSED

```

QUESTION 2:

What we want to do:

- Implement a **backtrace()** function inside xv6 kernel.
- backtrace() should walk the kernel stack and print the **return addresses (ra)** of active function calls.
- This helps debug where the kernel was executing (the call chain) when backtrace was invoked.
- Test it by writing a simple user program (bttest.c) that calls sleep(1), which enters the kernel and executes sys_sleep().
- Inside sys_sleep(), we add a call to backtrace(), so the stack trace is printed.

What I have done:

1) User space:

- user/bttest.c
 - A simple test program:
 - Calls sleep(1) so the kernel enters sys_sleep().
 - Prints "bttest: returned from sleep" when done.
 - Purpose: triggers backtrace inside the kernel.

Explaining bttest.c:

- #include "kernel/types.h", "user/user.h" → provide types and syscalls (like sleep, exit, printf).
- Main flow:
 1. Call sleep(1) → system call enters kernel.
 2. Kernel runs sys_sleep(), where we inserted backtrace().
 3. Backtrace prints call chain.
 4. Control returns → user program prints "bttest: returned from sleep".

2) Kernel space:

- kernel/riscv.h (already had helper, we used it)
 - r_fp() inline assembly returns the current frame pointer (s0).
 - Needed to walk stack frames.
- kernel/printf.c
 - Implemented backtrace(void) here because it needs printf.
 - Function logic:
 - Start from current frame pointer (uint64 fp = r_fp();).
 - While fp is within kernel stack range of this process:
 - Extract saved return address (ra = *((uint64*)(fp - 8))).
 - Print it with printf(" ra: 0x%p\n", (void*)ra);.
 - Move to previous frame pointer (fp = *((uint64*)(fp - 16))).
 - Stop when leaving valid stack.
- kernel/defs.h

- Added prototype:
- void backtrace(void);
- So other kernel files (like sysproc.c) can call it.
- kernel/sysproc.c
 - Modified sys_sleep():
 - Inserted backtrace(); at the beginning.
 - So whenever a process calls sleep, kernel prints its call chain.

3) Execution Flow with Example:

Input: \$ bttest

- Shell executes bttest:
- Loads the user program bttest.
- Inside bttest.c:
- Calls sleep(1) → triggers system call.
- Control switches from user space → kernel space.
- Inside kernel (sys_sleep):
- Kernel starts sys_sleep().
- We inserted a call to backtrace();.
- Inside backtrace():
- Reads current frame pointer using r_fp().
- Walks up the kernel stack frame by frame.
- At each frame:
 - Extracts saved return address (ra).
 - Prints it with printf(" ra: 0x%p\n", (void*)ra);
- Stops when leaving stack boundary.
- After backtrace:
- sys_sleep() continues waiting for ticks.
- When time passes, returns control to user program.
- User program resumes:
- Prints "bttest: returned from sleep".

4) Question and Answers:

- **Why only three outputs are coming, why can't more?**
 - The CPU switches from **user mode** → **kernel mode**.
 - The kernel always enters through **usertrap()** first.
 - Inside usertrap(), the kernel checks:
 - Was this an interrupt?
 - Or was it a system call?
 - If it's a system call, usertrap() calls **syscall()**.
 - syscall() looks at the syscall number (like "sleep = 13") and runs the correct kernel function (sys_sleep).
 - so that's why for sleep(1) we get 3 the flow is like usertrap() → syscall() → sys_sleep().
 - **so for remaining syscalls we can get more than 3 or less than three based on that syscall.**

OUTPUTS:

```
hart 1 starting
hart 2 starting
init: starting sh
$ bttest
backtrace:
  ra: 0x0x00000000080002998
  ra: 0x0x0000000008000286a
  ra: 0x0x000000000800025f2
bttest: returned from sleep
$ QEMU: Terminated
root@f17bfe48e7bc:/home/os-iitm/xv6-riscv# riscv64-unknown-elf-addr2line -f -C -e kernel/kernel
0x80002998
sys_sleep
/home/os-iitm/xv6-riscv/kernel/sysproc.c:66
0x8000286a
syscall
/home/os-iitm/xv6-riscv/kernel/syscall.c:173
0x800025f2
usertrap
/home/os-iitm/xv6-riscv/kernel/trap.c:80
```